Top 100 Databricks Interview Questions & Answers

Experienced / Senior Data Engineer Level

Contents

Architecture & Fundamentals (Q1-Q10)
Spark Core & Performance Tuning (Q11-Q30)
Delta Lake (Q31-Q50)
Structured Streaming & Ingestion (Q51-Q62)
Pipelines & Orchestration (Q63-Q70)
Unity Catalog & Governance (Q71-Q80)
Security, Networking & Administration (Q81-Q86)
DevOps & Engineering Practices (Q87-Q92)
SQL, ML & Scenario-Based (Q93-Q100)

Architecture & Fundamentals

Q1. Explain the Databricks architecture. What lives in the control plane vs. the data plane?

The **control plane** is managed by Databricks in its own account: web UI, notebooks, job scheduler, cluster manager, REST APIs and Unity Catalog metadata. The **data plane** is where data is actually processed. In *classic* compute it runs in your cloud account (VPC/VNet); in *serverless* it runs in a Databricks-managed account.

Your data stays in your own object storage (S3 / ADLS / GCS). With secure cluster connectivity, compute opens outbound connections to the control plane, so no inbound ports are needed.

Q2. How does Databricks differ from open-source Apache Spark?

OSS Spark is only the processing engine. Databricks adds a managed platform around it:

- Optimized Databricks Runtime (Photon, AQE defaults, disk cache, optimized Delta I/O)
- Managed clusters, autoscaling, serverless compute, SQL warehouses
- Delta Lake, Unity Catalog, Auto Loader, Lakeflow pipelines, Workflows
- Collaborative notebooks, MLflow, Model Serving, security and governance tooling

Q3. What is the Lakehouse architecture, and how does it differ from a data warehouse and a data lake?

A lakehouse puts warehouse capabilities (ACID transactions, schema enforcement, BI-grade performance, governance) directly on open-format files in cheap object storage (Delta/Parquet).

- **Warehouse:** structured data, proprietary storage, tightly coupled compute, weak for ML/unstructured data.
- **Data lake:** cheap and flexible, but no ACID, weak governance, often turns into a swamp.
- **Lakehouse:** one copy of data serving BI, streaming, data science and ML, with one governance layer (Unity Catalog).

Q4. Compare classic compute with serverless compute. What are the trade-offs?

Classic: runs in your account, so you control instance types, init scripts, libraries and networking. Start-up takes minutes. You pay cloud VM cost plus DBUs.

Serverless: run by Databricks, starts in seconds, no capacity planning, automatic upgrades and optimization. The price is a single bundled DBU rate, with less customization (limited init scripts and library control) and network access handled through network connectivity configs.

Choose serverless for bursty or low-ops workloads, classic for custom environments or when you want to optimize VM-level cost.

Q5. When would you use all-purpose clusters, job clusters, or SQL warehouses?

- **All-purpose clusters:** interactive development, exploration, shared notebooks. Higher DBU rate.
- **Job clusters:** created per job run and terminated afterwards; cheaper job-compute rate, isolated, the production default.
- **SQL warehouses:** BI dashboards, ad hoc SQL and JDBC/ODBC clients; Photon-enabled, with concurrency autoscaling (classic, pro, serverless types).

Q6. Explain the account, workspace, and metastore structure.

The **account** is the top-level entity for billing, identity (users, groups, service principals), and management of metastores and workspaces. A **workspace** is an environment containing compute, notebooks, jobs and dashboards. A **Unity Catalog metastore** is the governance container (one per cloud region) that holds catalogs, schemas and tables, and is attached to many workspaces so they share the same data and permissions.

Q7. What are the cluster modes, and when would you pick each?

- **Multi-node:** driver plus workers, for distributed workloads.
- **Single node:** driver only; good for small data, light ML, or tools that are not distributed.
- **Access modes:** *standard* (shared, UC-enforced isolation between users) and *dedicated* (assigned to one user or group, full language/library flexibility).

- **Autoscaling vs. fixed size:** autoscaling suits variable load; fixed size suits predictable, latency-sensitive jobs.

Q8. How do instance pools reduce cluster start-up time?

A pool keeps a set of idle, pre-provisioned cloud instances warm. Clusters acquire nodes from the pool instead of requesting new VMs, so they start in seconds. You pay the cloud provider for idle pool instances (no DBUs while idle). Pools help for frequent, short jobs; for long-running or infrequent jobs, serverless or plain job clusters are usually better.

Q9. What is the Photon engine, and when does it help?

Photon is Databricks' native, vectorized query engine written in C++. It accelerates SQL and DataFrame operations: scans, joins, aggregations, Delta reads/writes, and MERGE-heavy workloads. It gives the largest gains on large scans, joins and aggregations. It does not accelerate Python/Scala UDFs or RDD code. Photon has a higher DBU rate, but faster runtimes frequently lower total cost.

Q10. How do you manage Databricks Runtime version upgrades safely?

- Prefer LTS runtimes and pin the version explicitly in job and cluster definitions.
- Read release notes for Spark/behavior changes and deprecations.
- Upgrade in dev, then test, then prod, running regression tests and comparing outputs and performance.
- Roll out gradually (canary jobs first) and keep rollback easy by versioning configs in Asset Bundles or Terraform.

Spark Core & Performance Tuning

Q11. Explain lazy evaluation, the DAG, stages, and tasks.

Transformations (`select`, `filter`, `join`) only build a logical plan; nothing runs until an **action** (`count`, `write`, `collect`) triggers a job. Spark optimizes the plan with Catalyst, then builds a DAG of operations. The DAG is split into **stages** at shuffle boundaries, and each stage is split into **tasks**, one per partition, which run in parallel on executor cores. Lazy evaluation lets Spark optimize the whole pipeline end to end.

Q12. What is the difference between narrow and wide transformations?

Narrow: each output partition depends on a single input partition (`map`, `filter`, `select`), so no data movement is needed and operations pipeline within one stage.

Wide: output partitions depend on many input partitions (`groupBy`, `join`, `distinct`, `repartition`), which forces a **shuffle** (network and disk I/O) and a new stage boundary. Minimizing and tuning shuffles is the core of Spark performance work.

Q13. A Spark job is slow. Walk through how you'd diagnose it using the Spark UI.

1. **Jobs/Stages tab:** find the longest stages.
2. **Task metrics:** compare min/median/max task time. A large gap means skew.
3. **Shuffle read/write size, spill (memory/disk), and GC time** show memory pressure.
4. **SQL/DataFrame tab:** inspect the plan for join types, scan sizes, file/partition pruning and missing filters.
5. **Executors tab:** look for lost executors, uneven task distribution and OOM kills.
6. Fix the cause: skew, partition counts, broadcast hints, file layout, caching, or cluster sizing.

Q14. How do you detect and fix data skew?

Detect: a few tasks run far longer or process far more data than the median in the Spark UI.

Fix:

- Enable AQE skew join handling (`spark.sql.adaptive.skewJoin.enabled`).
- Broadcast the smaller side if it fits.
- Salt hot keys (append a random suffix, then aggregate in two steps).
- Isolate hot keys (e.g., nulls) and process them separately.
- Use a Databricks skew hint (`/*+ SKEW('t') */`) or repartition on a better key.

Q15. What is Adaptive Query Execution, and what does it optimize?

AQE re-optimizes the physical plan at runtime using actual shuffle statistics. It can: (1) **coalesce** small shuffle partitions, (2) **switch join strategy**, e.g. sort-merge to broadcast when one side turns out small, and (3) **split skewed partitions** in joins. It is enabled by default in Spark 3.2+ and Databricks, and reduces the need for manual tuning of `shuffle.partitions`.

Q16. Compare broadcast, sort-merge, and shuffle-hash joins.

- **Broadcast hash join:** one side is small (below `autoBroadcastJoinThreshold`) and is sent to all executors; no shuffle of the big side. Fastest.
- **Sort-merge join:** default for large equi-joins; both sides shuffled and sorted. Scales well but is expensive.
- **Shuffle hash join:** both sides shuffled, then a hash table is built on the smaller side per partition; no sort, but needs memory.

Non-equi joins fall back to broadcast nested loop or cartesian joins, which are costly.

Q17. What is the difference between `repartition` and `coalesce`?

`repartition(n)` does a **full shuffle**, can increase or decrease partitions, and balances data evenly (optionally by column). `coalesce(n)` only **decreases** partitions by merging existing ones without a full shuffle, so it is cheaper but can leave partitions uneven. Use `coalesce` to reduce output files; use `repartition` to fix skew or increase parallelism.

Q18. How do you choose the right number of shuffle partitions?

Default is 200, which is rarely right. Aim for roughly 100-200 MB of shuffle data per partition and at least 2-3x the total cores. Rely on AQE partition coalescing and Databricks auto-optimized shuffle, and only hard-tune for very large shuffles (raise it to avoid huge partitions and spill) or tiny ones (lower it to avoid task overhead).

Q19. Compare `cache` and `persist`. When can caching hurt performance?

`cache()` is `persist()` with the default storage level; `persist()` lets you choose the level (memory, disk, serialized, replicated). Caching hurts when the data is used only once, when it is larger than available memory (evictions and spill), when it holds stale data, or when it steals memory from execution. On Databricks, the separate **disk cache** transparently caches remote Parquet/Delta files on local SSDs and is often the better option.

Q20. How do you troubleshoot driver vs. executor out-of-memory errors?

Driver OOM: usually `collect()`, `toPandas()`, huge broadcasts, or too many partitions/files. Avoid pulling data to the driver, raise driver memory, limit results.

Executor OOM: caused by skew, large partitions, too many cores per executor (less memory per task), heavy UDFs or huge aggregations. Increase partitions, fix skew, use memory-optimized nodes, lower cores per executor. Confirm via executor logs and the Spark UI.

Q21. What is the small files problem, and how do you solve it?

Many tiny files cause slow listing, heavy metadata overhead, too many tasks and poor scan performance. Fixes: run `OPTIMIZE` to compact; enable **optimized writes** and **auto compaction**; `coalesce`/`repartition` before writes; use larger streaming trigger intervals; use Auto Loader batching; use liquid clustering; avoid over-partitioning.

Q22. What causes disk spill, and how do you reduce it?

Spill happens when execution memory is not enough for a shuffle, sort, join or aggregation, so Spark writes intermediate data to disk. Reduce it by increasing partitions (smaller per-task data), using memory-optimized instances or more memory per core, fixing skew, filtering and pruning early, selecting fewer columns, and enabling Photon.

Q23. Explain the Catalyst optimizer and Tungsten.

Catalyst is Spark SQL's rule-based and cost-based optimizer: analysis (resolve names/types), logical optimization (predicate pushdown, column pruning, constant folding, join reordering), physical planning (choosing strategies), and code generation.

Tungsten is the execution layer: binary off-heap memory management, cache-aware data structures and **whole-stage code generation**, which reduces JVM overhead and speeds up CPU-bound work.

Q24. Compare built-in functions, Python UDFs, and pandas UDFs.

- **Built-in functions:** run in the JVM and are fully optimizable by Catalyst/Photon. Always the first choice.
- **Python UDFs:** row-at-a-time, with serialization between JVM and Python; opaque to the optimizer; slow.
- **pandas (vectorized) UDFs:** use Apache Arrow to exchange batches, so far faster than Python UDFs, but still slower than built-ins.

Order of preference: built-ins, then SQL/Scala UDFs, then pandas UDFs, then plain Python UDFs.

Q25. What is dynamic partition pruning?

DPP uses a filter on a dimension table at runtime to prune partitions of a large partitioned fact table in a join, so Spark skips scanning irrelevant partitions. It requires the fact table to be partitioned on the join key and a selective filter on the dimension side. Databricks extends the idea to **dynamic file pruning** for Delta tables, which skips files based on min/max statistics.

Q26. Compare partitioning, Z-ordering, and liquid clustering.

- **Partitioning:** one directory per value; good for low-cardinality columns on very large tables; over-partitioning creates small files; keys are hard to change.
- **Z-ordering:** co-locates related data across multiple columns within files via `OPTIMIZE ... ZORDER BY`; must be re-run, and is not incremental.
- **Liquid clustering:** `CLUSTER BY (cols)` replaces both; keys can be changed without rewriting the table, clustering is incremental, and it handles high-cardinality and skewed columns. Recommended for new Delta tables.

Q27. How do you size executors (cores, memory, count)?

Rules of thumb: about 4-5 cores per executor (too many cores per JVM causes memory and I/O contention), enough memory per core for your largest partitions (typically 4-8 GB per core for general workloads), and total cores sized to input volume and SLA. Leave headroom for overhead. On Databricks, one executor typically maps to one worker node, so you mainly pick the instance type (memory- vs. compute-optimized) and worker count/autoscaling range.

Q28. What is speculative execution, and when would you enable it?

Spark re-launches slow-running tasks on other executors and takes whichever copy finishes first (`spark.speculation=true`). It helps with stragglers caused by a bad node or transient slowness. It does not fix data skew and can waste resources or cause duplicate side effects with non-idempotent writes, so use it carefully.

Q29. Explain broadcast variables and accumulators.

Broadcast variables ship a read-only value to each executor once (e.g., a lookup map) instead of with every task. **Accumulators** are write-only-from-tasks shared counters/sums aggregated on the driver (e.g., counting bad records). Accumulator updates inside transformations may be applied more than once on retries, so they are only reliable inside actions.

Q30. What are common performance pitfalls with window functions?

- Window without `PARTITION BY` pulls all data into a single partition.
- Skewed partition keys create straggler tasks.
- Multiple windows with different partitioning/ordering cause repeated shuffles and sorts.
- Large frames (`ROWS BETWEEN UNBOUNDED ...`) are memory-heavy.

Mitigate by partitioning on well-distributed keys, reusing identical window specs, filtering first, and using `row_number` + filter patterns carefully.

Delta Lake

Q31. How does Delta Lake provide ACID transactions on object storage?

Delta keeps a **transaction log** (`_delta_log`) of ordered JSON commit files (plus periodic Parquet checkpoints) recording add/remove file actions, metadata and protocol changes. A commit is atomic: a writer writes new data files, then commits by atomically creating the next log version. Readers build a consistent snapshot from the log. Concurrent writers use **optimistic concurrency control** with conflict checks at commit time.

Q32. Delta vs. Parquet: what do you gain?

Delta is Parquet files plus a transaction log. You gain ACID transactions, schema enforcement/evolution, time travel, `UPDATE`/`DELETE`/`MERGE`, data skipping via file statistics, unified batch and streaming reads/writes, compaction and clustering, change data feed, and constraints. Plain Parquet offers none of these.

Q33. Explain time travel, its use cases, and retention limits.

Query or restore earlier table versions with `VERSION AS OF`, `TIMESTAMP AS OF` or `RESTORE`. Use cases: audits, debugging, reproducing ML training data, rollback of bad writes. Limits are controlled by `delta.logRetentionDuration` (default 30 days) and `delta.deletedFileRetentionDuration` (default 7 days); once `VACUUM` removes data files, older versions can no longer be read.

Q34. What do `OPTIMIZE`, `VACUUM`, and `ZORDER` do? What are the risks of aggressive vacuuming?

`OPTIMIZE` compacts small files into larger ones (target roughly 1 GB). `ZORDER BY` (with OPTIMIZE) clusters data by columns for better data skipping. `VACUUM` physically deletes files no longer referenced and older than the retention period.

Risks of short retention: breaks time travel, can fail long-running queries or streams still reading old files, and breaks shallow clones. Do not go below 7 days without a clear reason.

Q35. When would you choose liquid clustering over partitioning?

Choose liquid clustering for high-cardinality filter columns, changing query patterns, skewed data, fast-growing tables, and tables that would otherwise be over-partitioned. Keep classic partitioning only when you have a very large table, a clear low-cardinality column (e.g., date) and a need for partition-level operations. Liquid clustering cannot be combined with partitioning or Z-order on the same table.

Q36. How would you implement SCD Type 2 using `MERGE INTO`?

Stage the source so each changed key produces two rows: one to **close** the current record and one to **insert** the new version.

```
MERGE INTO dim_customer t
USING (
  SELECT s.id AS merge_key, s.* FROM staged s
  UNION ALL
  SELECT NULL AS merge_key, s.*
  FROM staged s JOIN dim_customer d
    ON s.id = d.id AND d.is_current AND s.hash <> d.hash
) u
ON t.id = u.merge_key AND t.is_current
WHEN MATCHED AND t.hash <> u.hash THEN
  UPDATE SET is_current = false, end_date = current_date()
WHEN NOT MATCHED THEN
  INSERT (id, name, hash, start_date, end_date, is_current)
  VALUES (u.id, u.name, u.hash, current_date(), NULL, true)
```

In pipelines, `AUTO CDC` / `APPLY CHANGES` with SCD type 2 does this declaratively.

Q37. Compare schema enforcement and schema evolution.

Enforcement (default) rejects writes whose schema does not match the table, protecting data quality. **Evolution** allows controlled changes: `mergeSchema` on writes, `overwriteSchema`, `ALTER TABLE ADD COLUMNS`, or auto-merge for `MERGE`. Column mapping mode enables renaming and dropping columns without rewriting data.

Q38. What is Change Data Feed, and when would you use it?

CDF records row-level changes for a Delta table (`insert`, `update_preimage`, `update_postimage`, `delete`) with `_commit_version` and `_commit_timestamp`. Enable with `delta.enableChangeDataFeed = true`, read with `readChangeFeed`. Use it to propagate incremental changes to downstream tables, feed CDC to other systems, and audit changes without comparing full snapshots.

Q39. How does Delta handle concurrent writes? Explain optimistic concurrency and isolation levels.

Each writer works against a snapshot and, at commit, Delta checks whether another commit conflicted (e.g., modified or deleted the same files). If so, the commit fails with a concurrency exception and must be retried. Default isolation is **WriteSerializable** (stronger option: Serializable). Pure appends rarely conflict. Reduce conflicts by including partition/cluster predicates in `MERGE`/`UPDATE` conditions, and by using deletion vectors and liquid clustering, which support row-level concurrency.

Q40. What are deletion vectors, and what problem do they solve?

Deletion vectors mark deleted/updated rows in a small side file instead of rewriting whole Parquet files on `DELETE`, `UPDATE` and `MERGE`. This makes these operations much faster and reduces write amplification. Rewrites happen later during `OPTIMIZE`, and `REORG TABLE ... APPLY (PURGE)` forces physical removal (important for compliance).

Q41. Explain optimized writes and auto compaction.

Optimized writes shuffle data before writing so each task produces fewer, better-sized files (`delta.autoOptimize.optimizeWrite`). **Auto compaction** triggers a small compaction after a write if many small files remain (`delta.autoOptimize.autoCompact`). Together they prevent the small files problem without scheduling separate `OPTIMIZE` jobs for every table.

Q42. What role do checkpoints play in the Delta transaction log?

A checkpoint is a Parquet snapshot of the table state written periodically (every 10 commits by default). Readers load the latest checkpoint and replay only the JSON commits after it, instead of replaying the entire history. `_last_checkpoint` points to it. This keeps metadata reads fast for tables with long histories.

Q43. Compare shallow and deep clones.

Deep clone copies both metadata and data files, giving a fully independent table (backups, migration, DR; supports incremental re-sync). **Shallow clone** copies only metadata and references the source's data files: instant and nearly free, ideal for testing and experiments, but it depends on the source files (running `VACUUM` on the source can break it).

Q44. How would you recover a table after an accidental delete or bad update?

1. Inspect `DESCRIBE HISTORY` to identify the bad version.
2. `RESTORE TABLE t TO VERSION AS OF n` (or to a timestamp), provided files have not been vacuumed.
3. Alternatively, read the old version and `MERGE` back only the affected rows.
4. If a managed Unity Catalog table was dropped, use `UNDROP TABLE` within the retention window.

Add safeguards afterwards: restricted permissions, longer retention, and backups via deep clone.

Q45. Explain generated columns, identity columns, and constraints in Delta.

- **Generated columns:** values computed automatically from other columns (e.g., `event_date` from a timestamp), which can also enable partition pruning.
- **Identity columns:** `GENERATED ALWAYS AS IDENTITY` produces unique surrogate keys; values are unique but not guaranteed contiguous.
- **Constraints:** `NOT NULL` and `CHECK` constraints are enforced on write, so bad rows fail the transaction.

Q46. Which Delta table properties do you commonly tune, and why?

`delta.autoOptimize.optimizeWrite` / `autoCompact` (file sizing), `delta.targetFileSize` (file size), `delta.logRetentionDuration` and `delta.deletedFileRetentionDuration` (time travel vs. storage), `delta.enableChangeDataFeed`, `delta.enableDeletionVectors`, `delta.columnMapping.mode`, `delta.dataSkippingNumIndexedCols` (statistics columns, default 32), `delta.checkpointInterval`, and `delta.appendOnly` for immutable tables.

Q47. How do you handle late-arriving data and keep writes idempotent?

Make writes idempotent with `MERGE` on a business key, `replaceWhere` for partition-level overwrite, and `txnAppId`/`txnVersion` in `foreachBatch` so retries are not double-applied. Deduplicate on key plus ordering column (e.g., latest `updated_at`). For late data, use watermarks in streaming, merge with conditions like `s.updated_at > t.updated_at`, and reprocess affected partitions when needed.

Q48. Compare managed and external tables.

Managed tables: Unity Catalog manages both metadata and storage location; dropping the table deletes the data (managed tables can be undropped for a window), and they get optimizations like predictive optimization. **External tables:** data lives at a path you control; dropping removes only metadata. Prefer managed by default; use external when other systems must own or access the files directly.

Q49. What is Delta Sharing, and how does it work?

Delta Sharing is an open protocol for sharing live data securely across organizations and platforms without copying it. A provider creates a **share** (tables, views, volumes, notebooks, models) and grants it to **recipients**. Databricks-to-Databricks sharing is governed through Unity Catalog; open sharing uses a credential file/token, and the client reads data through short-lived pre-signed URLs. Works with Spark, pandas, Power BI, Tableau and others.

Q50. What is UniForm, and how does Delta interoperate with Iceberg?

UniForm (Universal Format) makes a Delta table automatically generate Iceberg (and Hudi) metadata alongside the Delta log, so Iceberg-compatible engines can read the same data files without copying. Enable via table property `delta.universalFormat.enabledFormats = 'iceberg'`. Unity Catalog can also expose an Iceberg REST catalog endpoint for external engines.

Structured Streaming & Ingestion

Q51. Explain the Structured Streaming model. How do micro-batch and continuous processing differ?

Structured Streaming treats a stream as an unbounded, continuously appended table; you write a normal DataFrame query and Spark runs it incrementally. **Micro-batch** (default) processes data in small batches at each trigger, with exactly-once guarantees and latency from about a hundred milliseconds up. **Continuous processing** (experimental) targets millisecond latency but offers only at-least-once semantics and supports limited operations.

Q52. Compare Auto Loader and `COPY INTO`. What are directory listing and file notification modes?

Auto Loader (`cloudFiles`) incrementally and scalably ingests new files, tracks progress in a checkpoint, infers and evolves schemas, and handles billions of files. `COPY INTO` is an idempotent SQL command for simpler, smaller-scale loads (thousands of files), without streaming semantics.

Auto Loader discovery modes: **directory listing** (default, uses incremental listing, no extra setup) and **file notification** (cloud event queues such as SQS/SNS, Event Grid or Pub/Sub; better for very large or deep directories).

Q53. How do checkpoints enable exactly-once semantics?

The checkpoint stores source offsets (write-ahead log), commit markers and operator state. On restart, Spark replays from the last committed offsets. Combined with a **replayable source** (Kafka, files) and an **idempotent sink** (Delta's transactional commits), each record's effect is applied exactly once.

Q54. Explain watermarking and how it handles late data.

A watermark = max event time seen minus an allowed delay (`withWatermark("ts", "10 minutes")`). Spark uses it to decide when a window is complete, emit results (append mode), and drop old state. Events older than the watermark are dropped. It bounds state size at the cost of ignoring very late records.

Q55. How does stateful streaming work? What is the RocksDB state store?

Operations such as aggregations, deduplication, stream-stream joins and `(flat)mapGroupsWithState` keep per-key **state** across batches, partitioned across tasks and checkpointed. The default state store keeps state in JVM memory, which can cause GC pressure at scale. The **RocksDB state store** keeps state in native memory and on local disk, supports millions of keys per node with stable latency, and is recommended for large state.

Q56. What trigger types exist, and when do you use each?

- **Default (unspecified):** next micro-batch starts as soon as the previous finishes; lowest latency.
- `processingTime="5 minutes"`**:** fixed interval; reduces small files and cost.
- `availableNow=True`**:** process everything available (in multiple batches) then stop; incremental batch on a schedule, very cost-efficient.
- `once`**:** deprecated in favor of `availableNow`.
- `continuous`**:** experimental low-latency mode.

Q57. How do stream-stream joins work, and what are their constraints?

Both sides buffer rows as state until matches arrive. To bound state, you need watermarks on both inputs and a time-range join condition (e.g., `b.ts BETWEEN a.ts AND a.ts + interval 1 hour`). Inner joins are supported with optional watermarks; outer joins require them, and unmatched rows are emitted only after the watermark passes, adding latency.

Q58. When would you use `foreachBatch`?

`foreachBatch` gives you each micro-batch as a regular DataFrame plus a batch ID. Use it to run `MERGE` upserts into Delta, write to multiple sinks, apply batch-only APIs, or add custom logic. For idempotency use the batch ID (or `txnAppId`/`txnVersion`) so retried batches do not duplicate writes.

Q59. How does Auto Loader handle schema evolution? What is the rescued data column?

Schema is inferred and stored at `cloudFiles.schemaLocation`. Evolution modes: `addNewColumns` (default; the stream fails once on a new column, updates the schema, and works after restart), `rescue` (never evolve; extra data goes to `_rescued_data`), `failOnNewColumns`, and `none`. The **rescued data column** captures data that does not fit the schema (new columns, type mismatches, case differences) so nothing is silently lost.

Q60. How do you ingest from Kafka, and how do you manage offsets and backpressure?

Use the `kafka` source with bootstrap servers, topic/subscribe options and security (SASL/SSL). Parse `key`/`value` (JSON, Avro with schema registry). Offsets are tracked in the Spark checkpoint, not committed to Kafka consumer groups. Control starting point with `startingOffsets`, backpressure with `maxOffsetsPerTrigger`, parallelism with `minPartitions`, and failure behavior with `failOnDataLoss`.

Q61. How do you design a streaming job to recover cleanly from failures?

Use a durable checkpoint location (unique per stream), idempotent sinks, and jobs configured with automatic retries or continuous-mode restarts. Avoid query changes that are incompatible with the existing checkpoint. Route malformed records to a dead-letter table, handle schema evolution explicitly, and monitor lag so you notice stalls early.

Q62. How do you monitor streaming lag and throughput?

Use the Spark UI Streaming tab and the `StreamingQueryListener` / `lastProgress` metrics: input rows per second, processed rows per second, batch duration, and offsets behind latest (e.g., `avgOffsetsBehindLatest` for Kafka). Compare input rate to processing rate; if processing is slower, lag grows. Export metrics to alerting systems and track event-time latency in the data itself.

Pipelines & Orchestration

Q63. What are Delta Live Tables / Lakeflow Declarative Pipelines, and what advantages do they offer?

A declarative framework: you define datasets and their queries, and the engine handles dependency resolution, orchestration, incremental processing, autoscaling, retries, error handling, data quality **expectations**, lineage and monitoring. It reduces boilerplate versus hand-written streaming and batch code and standardizes pipeline operations.

Q64. Compare streaming tables and materialized views.

Streaming tables process append-only sources incrementally (Auto Loader, Kafka, other streaming tables), each input row processed once; ideal for ingestion and low-latency transforms. **Materialized views** store the precomputed result of a query (joins, aggregations) and are refreshed, incrementally where possible, handling updates and deletes upstream; ideal for gold-layer aggregates and BI.

Q65. How do expectations work (warn, drop, fail)?

An expectation is a named SQL boolean constraint on a dataset.

- **Warn (default):** invalid rows are kept; violations are recorded as metrics.
- **Drop:** `ON VIOLATION DROP ROW` removes invalid rows.
- **Fail:** `ON VIOLATION FAIL UPDATE` stops the update.

Metrics are written to the pipeline event log for monitoring.

Q66. How do you implement CDC with `APPLY CHANGES INTO`?

Declare a target streaming table and apply a change feed with keys, a `SEQUENCE BY` column for ordering out-of-order events, `APPLY AS DELETE WHEN` for deletes, and `STORED AS SCD TYPE 1` or `2`. The engine handles upserts, ordering and history tracking. In newer releases this is exposed as `AUTO CDC` / `create_auto_cdc_flow`.

Q67. Compare triggered and continuous pipeline modes.

Triggered: each update processes available data and then stops; scheduled, cost-efficient, minutes-level latency. **Continuous:** the pipeline runs constantly with low latency, at the price of always-on compute. Choose by latency SLA and cost.

Q68. How would you design a medallion (bronze/silver/gold) architecture?

- **Bronze:** raw data ingested as-is (append-only, Auto Loader/Kafka) plus ingestion metadata; a replayable source of truth.
- **Silver:** cleansed, deduplicated, conformed and validated entities (expectations, CDC merges, joins).
- **Gold:** business-level aggregates and marts for BI/ML.

Organize with Unity Catalog (catalog per environment, schema per layer or domain), keep processing incremental and idempotent, define retention per layer, and apply clustering suited to each layer's access patterns.

Q69. Explain Databricks Workflows: task dependencies, retries, and repair runs.

A job is a DAG of tasks (notebooks, Python, SQL, pipelines, dbt, JAR, etc.) with dependencies, conditional logic (run-if, if/else), for-each loops, task and job parameters, timeouts, retries with backoff, notifications, and triggers (schedule, file arrival, table update, continuous). **Repair run** re-executes only failed tasks and their downstream dependents without rerunning successful ones.

Q70. When would you use Airflow instead of Databricks Workflows?

Use **Airflow** when you orchestrate many systems beyond Databricks, have an established Airflow platform, or need its ecosystem and complex cross-system scheduling. Use **Workflows** for Databricks-centric pipelines: no extra infrastructure, native integration with Unity Catalog, lineage, serverless, repair runs and lower operational cost. Airflow can still trigger Databricks jobs through its Databricks operators.

Unity Catalog & Governance

Q71. Explain the Unity Catalog object hierarchy.

Metastore, then catalog, then schema, then objects (tables, views, volumes, functions, ML models), giving a three-level namespace `catalog.schema.object`. Outside this chain, other securables include storage credentials, external locations, connections (federation), shares and recipients.

Q72. How do metastores, catalogs, and schemas map to environments or teams?

One metastore per cloud region, shared across workspaces. Common patterns: a catalog per environment (`dev`, `test`, `prod`) or per business domain, and schemas per medallion layer or subject area. Use workspace-to-catalog bindings to restrict which workspaces see which catalogs, and assign separate managed storage locations per catalog for isolation and cost tracking.

Q73. How do row filters and column masks work?

A **row filter** is a SQL function returning a boolean that is attached to a table and evaluated per query so users see only permitted rows (often using `is_account_group_member`). A **column mask** is a function that returns a transformed value for a column (e.g., masked email) based on the user's group. Both are enforced by Unity Catalog at query time, and newer attribute-based access control (ABAC) policies can apply them at scale using tags.

Q74. How do privileges, ownership, grants, and groups work in Unity Catalog?

Privileges (`USE CATALOG`, `USE SCHEMA`, `SELECT`, `MODIFY`, `CREATE TABLE`, `EXECUTE`, `READ VOLUME`, etc.) are granted on securables and **inherit downward**. Every object has an owner who can manage and grant on it. Best practice: grant to account-level **groups** (synced via SCIM), not individuals; use service principals for automation; follow least privilege.

Q75. What are storage credentials and external locations?

A **storage credential** wraps a cloud identity (IAM role, Azure managed identity, GCP service account) that Unity Catalog uses to access cloud storage. An **external location** binds a storage path to a credential and is the securable on which you grant `READ FILES` / `WRITE FILES` / `CREATE EXTERNAL TABLE`. They govern external tables, volumes and managed storage access.

Q76. How do you use lineage and audit logs?

Unity Catalog captures table- and column-level **lineage** automatically across notebooks, jobs, queries and dashboards; use it for impact analysis, root-cause debugging and compliance. **Audit logs** (`system.access.audit`) record logins, grants, data access and admin actions; use them for security reviews, anomaly detection and compliance evidence.

Q77. How would you migrate from the Hive metastore to Unity Catalog?

1. Inventory tables, ACLs, mounts, clusters, init scripts and jobs.
2. Create the metastore, storage credentials and external locations, and attach workspaces.
3. Migrate tables: `SYNC` or the upgrade tooling for external tables; `DEEP CLONE`/CTAS for managed tables.
4. Replace DBFS mounts with external locations and volumes.
5. Recreate permissions on account-level groups.
6. Update code to three-level names and move compute to UC-compatible access modes.
7. Test, run in parallel, then cut over. Hive metastore federation can ease the transition.

Q78. What are Volumes, and when do you use them?

Volumes are Unity Catalog objects that govern **non-tabular files** (managed or external) in cloud storage, accessed via `/Volumes/<catalog>/<schema>/<volume>/...`. Use them for landing raw files, ML artifacts, libraries and configs, replacing legacy DBFS mounts with governed access.

Q79. How do you manage secrets in Databricks?

Store credentials in **secret scopes** (Databricks-backed or backed by Azure Key Vault) and read them with `dbutils.secrets.get`; values are redacted in notebook output and access is controlled by ACLs. Prefer service principals with OAuth and Unity Catalog connections over long-lived tokens, never hard-code secrets, and manage scopes through Terraform/CI.

Q80. How do you design data isolation for multi-tenant or multi-environment setups?

Options, from strongest to lightest isolation: separate workspaces and metastores/catalogs per tenant or environment; catalogs per tenant with workspace bindings and separate storage credentials/locations; shared tables with row filters and column masks. Add per-tenant service principals, cluster policies, tags for cost attribution and network isolation as required. The trade-off is isolation versus operational overhead and cost.

Security, Networking & Administration

Q81. Explain VNet injection, Private Link, and secure cluster connectivity.

VNet/VPC injection deploys classic compute inside your own virtual network so you control subnets, security groups and routing. **Private Link** provides private connectivity for front-end access (users and clients to the workspace) and back-end access (compute to the control plane) without traversing the public internet. **Secure cluster connectivity** (no public IPs) means compute has no open inbound ports and initiates outbound connections to the control plane.

Q82. How do you set up SSO, SCIM, and service principals?

Configure **SSO** (SAML/OIDC) with your identity provider at the account level. Enable **SCIM** provisioning from the IdP (Entra ID, Okta) so users and groups sync automatically. Create **service principals** for automation (jobs, CI/CD), authenticate with OAuth machine-to-machine credentials, add them to groups, and run production jobs as the service principal rather than as a person.

Q83. How do cluster policies help with governance and cost control?

Policies restrict and default cluster settings: allowed instance types and runtime versions, max workers and DBUs per hour, mandatory auto-termination, required tags for chargeback, library and init script rules. Assign them to groups so users self-serve compute within guardrails.

Q84. What strategies do you use for cost optimization?

- Use job clusters or serverless for jobs; shut down idle all-purpose clusters (auto-termination).
- Autoscaling and right-sized instances; spot/preemptible workers for fault-tolerant work.
- Photon where it shortens runtime; instance pools for frequent short jobs.
- Efficient data layout (compaction, clustering, data skipping) to reduce I/O.
- Cluster policies, tags and budgets for visibility; commit-based discounts for steady usage; monitor via system tables.

Q85. How do you use system tables for monitoring and billing analysis?

Query `system.billing.usage` (with `list_prices`) for cost by workspace, SKU, tag, job or warehouse; `system.compute.*` for cluster and node utilization; `system.query.history` for query performance; `system.lakeflow.*` for job and pipeline runs; and `system.access.audit` for security events. Build dashboards and alerts on top, such as spend spikes, idle clusters and failing jobs.

Q86. How would you design disaster recovery for a Databricks environment?

Define RPO/RTO first. Typical active-passive design: a secondary-region workspace deployed from infrastructure as code (Terraform/Asset Bundles); data replicated via cloud storage replication or scheduled Delta deep clones; identities via SCIM; secrets, cluster policies and jobs synced from source control; Unity Catalog metadata recreated in the secondary region. Plan how streaming checkpoints and in-flight work are handled, and rehearse failover regularly.

DevOps & Engineering Practices

Q87. How do you implement CI/CD on Databricks?

Keep code in Git (Git folders). **Databricks Asset Bundles** define jobs, pipelines and clusters in YAML with per-environment targets. CI (GitHub Actions, Azure DevOps, etc.) runs linting and unit tests, then `databricks bundle validate` and `deploy` to staging, runs integration tests, then promotes to prod using a service principal. **Terraform** manages workspace-level infrastructure (workspaces, Unity Catalog, networking, permissions).

Q88. How do you unit and integration test PySpark code?

Structure logic as pure functions that take and return DataFrames. Unit-test them with `pytest` and a small local SparkSession (compare results with helpers like chispa). Run integration tests against a test workspace through Databricks Connect or `bundle run` with sample data, and add data-quality checks. Run everything in CI on each pull request.

Q89. How do you structure dev, test, and prod environments?

Separate workspaces (or at least separate catalogs) per environment. Dev uses personal or shared dev schemas with small data; test/staging mirrors production and runs automated deployments; prod is locked down and changed only by CI/CD service principals. Parameterize catalog names and configs via bundle targets so the same code runs everywhere.

Q90. How do you parameterize notebooks and jobs?

Notebooks use **widgets** (`dbutils.widgets`). Jobs use **job and task parameters** with dynamic value references (e.g., `{{job.parameters.env}}`), `dbutils.jobs.taskValues` to pass values between tasks, and bundle variables per target. Avoid hard-coded paths or catalog names.

Q91. What is Databricks Connect, and when is it useful?

Databricks Connect lets you run Spark code from a local IDE or application against remote Databricks compute (built on Spark Connect in modern versions). Use it for IDE development and debugging, running integration tests from CI, and embedding Spark in applications. Some APIs (such as RDDs) are not supported.

Q92. What Git workflow and branching strategy do you use with Databricks?

Use Git folders linked to the remote repo. Developers work on feature branches in personal Git folders, open pull requests with code review and CI checks, merge to `main` (auto-deploy to staging), and promote to production with release tags or a gated pipeline. Never edit directly in production; store notebooks in source format for readable diffs.

SQL, ML & Scenario-Based

Q93. How do you size Databricks SQL warehouses? Compare serverless and pro, and explain query caching.

Warehouse **size** affects per-query speed; **min/max clusters** affect concurrency. Start small, load test, then scale. **Serverless** starts in seconds, uses intelligent workload management and predictive I/O, and is best for bursty BI. **Pro** has Photon and predictive I/O with slower start-up and no serverless workload management; **classic** is the most basic. Caching layers: **result cache** for identical queries, and the **disk cache** that keeps hot remote data on local SSDs.

Q94. Explain the MLflow components.

- **Tracking:** logs parameters, metrics and artifacts per run within experiments.
- **Projects:** reproducible packaging of code and environment.
- **Models:** a standard packaging format (flavors) for deployment.
- **Model Registry:** versioned models with aliases, lineage and governance (registered in Unity Catalog).

Newer features add evaluation and tracing for LLM/GenAI applications.

Q95. How do Feature Store and Model Serving fit into a production ML workflow?

Feature tables in Unity Catalog give consistent, reusable features with point-in-time lookups for training and automatic lookup at inference. Models are logged and versioned in MLflow/Unity Catalog, then deployed on **Model Serving** (serverless REST endpoints with autoscaling and traffic splitting). Workflows handle retraining, and monitoring tracks drift and quality.

Q96. Design an end-to-end pipeline that ingests 10 TB of data per day.

10 TB/day is roughly 115 MB/s sustained, which is manageable with proper design.

- **Ingest:** Auto Loader (file notification mode) or Kafka into bronze, with optimized writes and right-sized files.
- **Process:** incremental silver/gold with Lakeflow pipelines or Structured Streaming `availableNow`, on autoscaling Photon job clusters or serverless.
- **Layout:** liquid clustering on common filter keys (such as date plus an entity key).
- **Reliability:** idempotent writes, expectations, dead-letter handling, schema evolution via rescued data.
- **Operations:** monitoring, alerts, cost tags, retention and `OPTIMIZE`/`VACUUM` policies (or predictive optimization).

Q97. How would you migrate an on-prem Hadoop or Teradata workload to Databricks?

1. **Assess:** inventory data, jobs, dependencies, SQL dialects and SLAs.
2. **Land data:** bulk copy to cloud storage (distcp/transfer tools; JDBC/export or federation for Teradata) and convert to Delta.
3. **Convert code:** HiveQL/Spark and Teradata SQL to Spark SQL/Databricks SQL, using migration tooling where available.
4. **Rebuild orchestration:** Oozie/Control-M to Workflows; Ranger policies to Unity Catalog.
5. **Validate:** row counts, checksums and reconciliation; run old and new in parallel.
6. **Cut over,** then optimize layout and cost.

Q98. A pipeline suddenly runs 5x slower than yesterday. How do you investigate?

Establish what changed: input volume or skew, code or config deploys, runtime version, cluster size or spot loss, table layout (small files, missed `OPTIMIZE`), concurrent workloads, or upstream delays. Compare Spark UI stages and query plans with a previous good run (look for join strategy changes, spill, skew, scan sizes), check `DESCRIBE HISTORY` and system tables, fix the root cause, and add alerting on runtime regressions.

Q99. How would you implement GDPR "right to be forgotten" on Delta tables?

Locate the subject's data across tables (lineage, PII tags, key mapping), then `DELETE FROM t WHERE user_id = ...`. Because time travel and deletion vectors keep old data physically present, run `REORG TABLE ... APPLY (PURGE)` where needed and `VACUUM` after the retention window. Handle downstream tables, change feeds, backups, checkpoints and logs. Batch deletion requests to limit rewrites, cluster by user key for efficient deletes, and keep an audit trail. Pseudonymization or crypto-shredding is an alternative design.

Q100. How would you design a combined batch and real-time architecture, and how do you decide between Lambda and Kappa styles?

Lambda: separate batch and speed layers merged at serving; accurate and flexible but duplicates logic and increases operational burden. **Kappa:** a single streaming pipeline, with reprocessing by replaying the log.

On Databricks, Delta serves as unified batch and streaming storage, and the same code can run with Structured Streaming continuously (real-time) or with `availableNow` (scheduled batch), so a **Kappa-like medallion** design is usually the default. Decide based on latency SLAs, replayability of sources (e.g., Kafka retention), need for heavy full-history batch logic, team skills and cost. A hybrid is common: streaming for freshness plus periodic batch for backfills and corrections.